# Tugas Besar 1 IF3070 Dasar Inteligensi Artifisial
# Penempatan Kendaraan pada Geladak Kapal dengan Algoritma Local Search




Kelompok 41

| | |
|---|---|
| Amanda Trishya Aulia | / 18224020 |
| Mineva Azzahra | / 18224029 |
| Adriel Nathanael S. | / 18224109 |

**Program Studi Sistem dan Teknologi Informasi**
**Sekolah Teknik Elektro dan Informatika**
**Institut Teknologi Bandung**


**Semester I 2026/2027**

# Daftar Isi

**Daftar Isi** ..... **2**
**BAB 1 Deskripsi Persoalan** ..... **3**
1.1 Latar Belakang ..... 3
1.2 Deskripsi Masalah ..... 3
1.3 Objective Function dan Constraints ..... 3
1.4 Batasan dan Asumsi ..... 4
**BAB 2 Pembahasan** ..... **5**
2.1 Representasi Permasalahan ..... 5
2.1.1 Representasi State ..... 5
2.1.2 Initial State (Random) ..... 6
2.1.3 Fungsi Objektif dan Penanganan Constraint ..... 7
2.1.4 Neighbor / Move as ..... 8
2.2 Implementasi Algoritma ..... 10
2.2.1 Hill-Climbing ..... 10
2.2.2 Simulated Annealing ..... 12
2.2.3 Genetic Algorithm ..... 14
2.2.4 Visualisasi ..... 18
2.3 Hasil dan Analisis Eksperimen ..... 21
2.3.1 Setup Eksperimen ..... 21
2.3.2 Hill-Climbing: Steepest Ascent ..... 22
2.3.3 Simulated Annealing ..... 23
2.3.4 Genetic Algorithm ..... 25
2.3.5 Analisis Perbandingan Antar Algoritma ..... 33
**BAB 3 Kesimpulan dan Saran** ..... **35**
3.1 Kesimpulan ..... 35
3.2 Saran ..... 35
**BAB 4 Pembagian Tugas** ..... **36**
**Referensi** ..... **37**
**Lampiran** ..... **38**

# BAB 1
# Deskripsi Persoalan

## 1.1 Latar Belakang

PT Owen Intim Line merupakan perusahaan pelayaran yang melayani pengangkutan kendaraan menggunakan kapal roro/kargo. Dalam operasional sehari-hari, penataan dan penempatan posisi kendaraan (*vehicle packing/placement*) di atas geladak kargo kapal masih sering dilakukan secara manual atau berbasis heuristik sederhana.

Penataan manual ini sering kali menghasilkan pemanfaatan ruang geladak yang tidak optimal, keterbatasan kapasitas angkut total, serta berisiko memicu ketidaksesuaian urutan bongkar-muat.. Untuk mengatasi kendala tersebut, diperlukan suatu pendekatan komputasional berbasis Local Search dan Evolutionary Algorithm (seperti Hill Climbing, Simulated Annealing, dan Genetic Algorithm) untuk mengoptimalkan penataan kendaraan pada geladak kapal secara otomatis.

## 1.2 Deskripsi Masalah

Permasalahan yang diselesaikan adalah menentukan konfigurasi penempatan kendaraan pada geladak kapal kargo, yaitu kendaraan mana yang diangkut, di koordinat mana, dan dengan orientasi apa. Geladak dan setiap kendaraan dimodelkan sebagai grid 2D berbentuk persegi panjang. Kendaraan yang tidak terangkut berada di luar kapal.

Entitas dan atribut yang digunakan:

- Vehicle: id, dimensi (width, length), orientasi (Horizontal/Vertical), shippingFee (juta rupiah), weight (unit), dan eta.
- Ship: dimensi geladak (width, length) dan maxCapacity (unit).

Tujuan permasalahan adalah memaksimalkan keuntungan pengiriman kapal sekaligus menjaga urutan penempatan agar sesuai dengan urutan kedatangan (ETA) kendaraan.

## 1.3 Objective Function dan Constraints

Objective function. Nilai sebuah state s dihitung dengan:

```
F(s) = Σ ShippingFee(v) − w · EtaPenalty(s)
```

- **ΣShippingFee(v)** adalah total tarif seluruh kendaraan v yang berada di dalam kapal (kendaraan di luar kapal tidak dihitung).
- **w = ETA_WEIGHT = 3.**
- **EtaPenalty(s)** adalah jumlah pasangan kendaraan terangkut (a, b) yang urutannya terbalik, yaitu ETA a < ETA b tetapi koordinat y a > y b. Asumsinya, kendaraan dengan ETA lebih kecil harus berada di y yang lebih kecil (lebih dekat ke sisi bongkar), sehingga tidak terhalang kendaraan lain saat dibongkar.

## 1.4 Batasan dan Asumsi

1. **Koordinat Bilangan Bulat:** Geladak kapal dan dimensi kendaraan direpresentasikan sebagai grid 2 dimens. Koordinat posisi (x,y) seluruhnya berupa **bilangan bulat (*integer*)**.
2. **Penataan 2D:** Penempatan kendaraan hanya dilakukan secara 2 dimensi pada permukaan geladak kapal (tidak memperhitungkan penumpukan secara vertikal/3D).
3. **Fleksibilitas Pemuatansa:** Tidak semua kendaraan wajib dimasukkan ke dalam kapal. Kendaraan dapat dikeluarkan atau diletakkan di luar kapal.
4. **Karakteristik Orientasi:** Kendaraan hanya dapat memiliki 2 pilihan orientasi, yaitu 'Horizontal' dan 'Vertical'.

# BAB 2
# Pembahasan

## 2.1 Representasi Permasalahan

### 2.1.1 Representasi State

Satu state merepresentasikan satu solusi, yaitu penempatan semua kendaraan (di dalam atau di luar kapal) pada geladak. Representasi memakai tiga kelas: Vehicle, Ship, dan State.

Kelas Vehicle menyimpan data satu kendaraan, dengan atribut:

- id: identitas kendaraan.
- width, length: ukuran kendaraan dalam satuan grid.
- orientation: 'Horizontal' atau 'Vertical'.
- shippingFee, weight, eta: tarif, berat, dan estimasi waktu kedatangan.
- x, y: koordinat penempatan di geladak. Bernilai None jika kendaraan berada di luar kapal.

```
class Vehicle:
    def __init__(self, id, width, length, shippingFee, weight, eta):
        self.id = id
        self.width = width
        self.length = length
        self.orientation = 'Horizontal'
        self.shippingFee = shippingFee
        self.weight = weight
        self.eta = eta
        self.x = None
        self.y = None

    def rotate(self):
        if self.orientation == 'Horizontal':
            self.orientation = 'Vertical'
        else:
            self.orientation = 'Horizontal'

    def getCurrentDimensions(self):
        if self.orientation == 'Horizontal':
            return self.width, self.length
```

```
        return self.length, self.width
```

Satu state merepresentasikan satu solusi, yaitu penempatan semua kendaraan (di dalam atau di luar kapal) pada geladak. Representasi memakai tiga kelas: Vehicle, Ship, dan State.

rotate( ) membalik orientasi kendaraan. getCurrentDimensions( ) mengembalikan ukuran kendaraan sesuai orientasinya (lebar dan panjang ditukar jika Vertical).

Kelas Ship menyimpan ukuran geladak dan kapasitas maksimum kapal.

```
class Ship:
    def __init__(self, width, length, maxCapacity):
        self.width = width
        self.length = length
        self.maxCapacity = maxCapacity
```

Kelas State menyimpan kapal (ship) dan daftar seluruh kendaraan (vehicles). Daftar ini berisi semua kendaraan, baik yang di dalam maupun di luar kapal.

```
class State:
    def __init__(self, ship, vehicles):
        self.ship = ship
        self.vehicles = copy.deepcopy(vehicles) if vehicles else []

    def clone(self):
        return State(self.ship, self.vehicles)
```

__init__( ) membuat state dengan menyalin daftar kendaraan, supaya perubahan pada satu state tidak memengaruhi state lain. clone( ) membuat salinan state yang dipakai saat mencoba langkah baru tanpa merusak state asli.

### 2.1.2 Initial State (Random)

```
   @staticmethod
    def generateRandomState(ship, vehicles):
        state = State(ship, vehicles)
        urutan = list(range(len(state.vehicles)))
        random.shuffle(urutan)
        for i in urutan:
            v = state.vehicles[i]
```

```
            for _ in range(20):
                v.orientation = random.choice(['Horizontal',
'Vertical'])
                w, l = v.getCurrentDimensions()
                if ship.width >= w and ship.length >= l:
                    v.x = random.randint(0, ship.width - w)
                    v.y = random.randint(0, ship.length - l)
                    if state.isValid():
                        break
                v.x = None
                v.y = None
        return state
```

Fungsi ini membuat state awal secara acak. Urutan kendaraan diacak, lalu setiap kendaraan dicoba ditempatkan paling banyak 20 kali dengan orientasi dan koordinat acak. Jika penempatan valid, kendaraan dipertahankan di situ. Jika setelah 20 percobaan tidak ada yang valid, kendaraan dibiarkan di luar kapal. Dengan cara ini state awal selalu valid.

### 2.1.3 Fungsi Objektif dan Penanganan Constraint

```
   def getShippingFee(self):
        return sum(v.shippingFee for v in self.vehicles if v.x is not
None)

    def getEtaPenalty(self):
        placed = [v for v in self.vehicles if v.x is not None]
        penalty = 0
        for i in range(len(placed)):
            for j in range(i + 1, len(placed)):
                a, b = placed[i], placed[j]
                if (a.eta < b.eta and a.y > b.y) or (b.eta < a.eta and
b.y > a.y):
                    penalty += 1
        return penalty

    def getFitness(self):
        return self.getShippingFee() - 3 * self.getEtaPenalty()
```

getShippingFee( ) menjumlahkan tarif semua kendaraan yang berada di dalam kapal. getEtaPenalty( ) menghitung banyak pasangan kendaraan di kapal yang ETA-nya lebih awal tetapi ditempatkan di

koordinat y lebih besar. getFitness( ) menghitung nilai akhir. Nilai fitness yang lebih besar berarti solusi lebih baik, tujuannya memaksimalkan ini.

Penanganan constraint dilakukan dengan fungsi isValid( )

```
   def isValid(self):
        grid = [[False] * self.ship.length for _ in
range(self.ship.width)]
        berat = 0
        for v in self.vehicles:
            if v.x is None or v.y is None:
                continue
            berat += v.weight
            w, l = v.getCurrentDimensions()
            if berat > self.ship.maxCapacity:
                return False
            if v.x < 0 or v.y < 0 or v.x + w > self.ship.width or v.y +
l > self.ship.length:
                return False
            for i in range(v.x, v.x + w):
                for j in range(v.y, v.y + l):
                    if grid[i][j]:
                        return False
                    grid[i][j] = True
        return True
```

Fungsi ini memeriksa tiga batasan: total berat tidak melebihi kapasitas kapal, seluruh sel kendaraan berada di dalam geladak, dan tidak ada sel yang ditempati lebih dari satu kendaraan. Jika salah satu dilanggar, fungsi mengembalikan False.

State yang tidak valid ditolak. Pada Hill Climbing dan Simulated Annealing, tetangga yang tidak valid tidak dipakai. Pada Genetic Algorithm, hasil *crossover* dan mutasi yang melanggar batasan diperbaiki dengan mengeluarkan kendaraan yang bermasalah dari kapal.

**Bonus Objective:** kelompok kami menambahkan penalti ETA sebagai heuristik agar kendaraan dengan ETA lebih awal ditempatkan pada y yang lebih kecil. Bobot penalti diatur sebesar 3, dan digabung dengan Fitness function untuk penggunaannya

### 2.1.4 Neighbor / Move as

Langkah neighbor yang digunakan:

1. Pindah kendaraan ke koordinat lain di geladak.
2. Ubah orientasi kendaraan yang berada di kapal.
3. Tukar dua kendaraan, baik di dalam maupun di luar kapal.

```
def swapVehiclePositions(self, a, b):
    va = self.vehicles[a]
    vb = self.vehicles[b]
    va.x, vb.x = vb.x, va.x
    va.y, vb.y = vb.y, va.y

def moveVehicle(self, i, x, y):
    self.vehicles[i].x = x
    self.vehicles[i].y = y

def rotateVehicle(self, i):
    self.vehicles[i].rotate()

def getMoves(self):
    moves = []
    for i, v in enumerate(self.vehicles):
        for x in range(self.ship.width):
            for y in range(self.ship.length):
                if (v.x, v.y) != (x, y):
                    moves.append(("move", i, x, y))
        if v.x is not None:
            moves.append(("rotate", i))
        for j in range(i + 1, len(self.vehicles)):
            if v.x is not None or self.vehicles[j].x is not None:
                moves.append(("swap", i, j))
    return moves

def applyMove(self, move):
    s = self.clone()
    if move[0] == "move":
        s.moveVehicle(move[1], move[2], move[3])
    elif move[0] == "rotate":
        s.rotateVehicle(move[1])
    else:
        s.swapVehiclePositions(move[1], move[2])
    return s
```

## 2.2 Implementasi Algoritma

### 2.2.1 Hill-Climbing

Hill Climbing adalah algoritma *local search* dasar yang bersifat iteratif. Algoritma ini dapat dianalogikan seperti seorang pendaki yang mencoba mencapai puncak (atau lembah) dengan kondisi pandangan terbatas. Pada setiap langkah, pendaki hanya akan bergerak ke state tetangga yang terlihat "naik" (atau "turun" untuk minimasi) dari posisinya saat ini.

#### 2.2.1.1 Hill-Climbing: Steepest Ascent

Varian ini adalah varian Hill Climbing yang paling *greedy*. Pada setiap iterasi, algoritma akan mengevaluasi semua *state* tetangga yang mungkin. Ia kemudian akan memilih untuk pindah ke *state* tetangga yang memberikan perbaikan paling bagus. Kalau tidak ada tetangga yang memiliki skor lebih baik daripada state saat ini, yaitu ketika skornya sama atau lebih buruk, algoritma akan langsung berhenti, karena ia telah sampai di local optimum.

- **Kelas HillClimbing**

```
class HillClimbing:
    def __init__(self, ship, vehicles):
        self.ship = ship
        self.vehicles = vehicles
        self.initialState = None
        self.history = []
        self.iterations = 0
        self.duration = 0
```

Konstruktor menyimpan masukan masalah (ship dan vehicles) serta data hasil pencarian. initialState untuk visualisasi state awal, *history* berisi *fitness* tiap state yang dilewati (untuk plot objective function), dan *iterations* serta duration untuk banyak iterasi dan durasi pencarian.

- **Fungsi getNeighbors**

```
   def getNeighbors(self, current):
        neighbors = []
        for move in current.getMoves():
```

```
            s = current.applyMove(move)
            if s.isValid():
                neighbors.append(s)
        return neighbors
```

Fungsi ini menghasilkan semua tetangga dari state saat ini. getMoves() menyusun semua langkah: pindah koordinat, putar orientasi, dan tukar dua kendaraan. Setiap langkah diterapkan ke salinan state, lalu hanya yang lolos isValid() tidak bertumpuk, di dalam geladak, tidak melebihi kapasitas yang disimpan. Jumlah tetangganya bisa mencapai ratusan, sehingga pencarian Steepest Ascent memakan waktu cukup lama.

- **Fungsi runSteepestAscent**

```
   def runSteepestAscent(self):
        mulai = time.perf_counter()
        current = State.generateRandomState(self.ship, self.vehicles)
        self.initialState = current.clone()
        self.history = [current.getFitness()]

        while True:
            tetangga = self.getNeighbors(current)
            if len(tetangga) == 0:
                break
            best = max(tetangga, key=lambda s: s.getFitness())
            if best.getFitness() <= current.getFitness():
                break
            current = best
            self.history.append(current.getFitness())

        self.iterations = len(self.history) - 1
        self.duration = time.perf_counter() - mulai
        return current
```

Fungsi ini adalah intinya. Pencarian dari state awal acak disimpan di initialState. Di loop, ambil semua tetangga dan pilih yang fitness-nya paling tinggi. Jika tetangga terbaik tidak lebih baik dari state saat ini, loop berhenti karena sudah di local optimum. Jika lebih baik, state saat ini diganti dan fitness-nya dicatat ke history. Setelah selesai, iterations diisi jumlah perpindahan yang dilakukan, duration diisi lama pencarian, dan state akhir dikembalikan.

- **Fungsi getFitness**

```
    def getFitness(self):
        return self.getShippingFee() - 3 * self.getEtaPenalty()
```

Semakin tinggi nilai *fitness*, semakin baik konfigurasi penempatan yang dihasilkan. Algoritma berusaha memaksimalkan pendapatan pengiriman dengan mempertimbangkan urutan ETA.

### 2.2.2 Simulated Annealing

Simulated Annealing (SA) adalah algoritma local search yang terinspirasi dari proses pengeringan/pendinginan logam (*annealing*) dalam ilmu metalurgi. Algoritma ini dirancang untuk mengatasi kelemahan utama Hill-Climbing yang mudah terjebak di local optimum. Berbeda dengan Hill-Climbing yang bersifat murni *greedy*, Simulated Annealing mengizinkan langkah yang memperburuk nilai fitness dengan probabilitas tertentu. Probabilitas penerimaan langkah buruk ini bergantung pada parameter suhu (T) dan besarnya penurunan nilai fitness (Delta E). Di awal iterasi saat suhu masih tinggi, algoritma sangat eksploratif sehingga dapat keluar dari local optimum. Seiring berkurangnya suhu secara bertahap (cooling schedule), algoritma menjadi lebih selektif dan berkonvergensi seperti Hill-Climbing.

- **Kelas SimulatedAnnealing**

Kelas ini mengatur parameter utama pencarian seperti suhu awal (*temperature*), laju pendinginan (*cooling rate*), serta memantau riwayat pencarian

```
class SimulatedAnnealing:
    def __init__(self, ship, vehicles, iterations=500,
temperature=100.0, cooling=0.99):
        self.initialState = State.generateRandomState(ship, vehicles)
        self.iterations = iterations
        self.temperature = temperature
        self.cooling = cooling
        self.history = []
        self.current_history = []
        self.acceptance_history = []
        self.stuck_count = 0
        self.duration = 0
```

- **Fungsi randomNeighbor**

Untuk efisiensi pencarian, fungsi ini memilih satu langkah (move) acak dari himpunan tetangga yang valid. Langkah dikelompokkan berdasarkan jenisnya (swap, move, rotate), kemudian satu jenis dan langkah dipilih secara acak.

```
def randomNeighbor(self, state):
        groups = {}
        for move in state.getMoves():
            groups.setdefault(move[0], []).append(move)
        if len(groups) == 0:
            return state.clone()
        for _ in range(500):
            kind = random.choice(list(groups))
            s = state.applyMove(random.choice(groups[kind]))
            if s.isValid():
                return s
        return state.clone()
```

- **Fungsi run**

Fungsi **run()** pada kelas SimulatedAnnealing dieksekusi untuk mengontrol seluruh alur pencarian konfigurasi penempatan kendaraan secara iteratif. Proses diawali dengan pembentukan state awal acak yang valid, yang kemudian disalin sebagai solusi awal sekaligus solusi terbaik sementara, serta penentuan suhu awal (temperature). Pada setiap iterasi, algoritma memilih satu tetangga acak dari state saat ini melalui fungsi randomNeighbor() dan menghitung selisih nilai fitness-nya *(Delta E)*. Kandidat tetangga yang menghasilkan fitness lebih tinggi akan langsung diterima. Sementara itu, kandidat dengan fitness lebih rendah tidak langsung ditolak, melainkan dievaluasi menggunakan rumus *(e^Delta E / T)*. Mekanisme penerimaan solusi buruk dengan probabilitas ini memungkinkan algoritma keluar dari jebakan local optimum. Setiap kali solusi baru yang lebih unggul ditemukan, variabel solusi terbaik (best) akan diperbarui. Di akhir tiap iterasi, suhu T diturunkan secara bertahap menggunakan faktor pendinginan (variabel cooling), yang secara perlahan membuat algoritma menjadi lebih selektif hingga berkonvergensi pada solusi akhir yang optimal.

```
def run(self):
    mulai = time.perf_counter()
    current = self.initialState.clone()
    best = current.clone()
    T = self.temperature
```

```
    self.history = [best.getFitness()]
    self.current_history = [current.getFitness()]
    self.acceptance_history = []
    self.stuck_count = 0
    same_score = 0

    for i in range(self.iterations):
        candidate = self.randomNeighbor(current)
        delta = candidate.getFitness() - current.getFitness()
        prob = 1.0 if delta >= 0 else math.exp(delta / T)

        if random.random() < prob:
            current = candidate

        if current.getFitness() > best.getFitness():
            best = current.clone()
            same_score = 0
        else:
            same_score += 1
            if same_score == 10:
                self.stuck_count += 1

        self.history.append(best.getFitness())
        self.current_history.append(current.getFitness())
        self.acceptance_history.append(prob)
        T = T * self.cooling

    self.duration = time.perf_counter() - mulai
    return best
```

### 2.2.3 Genetic Algorithm

Genetic Algorithm merupakan algoritma *local search* yang berkembang dari algoritma Hill-Climbing Random Restart. Cara kerja algoritma ini dimulai dengan *generate* beberapa *complete state* secara *random* dan menghitung masing-masing nilai *fitness* mereka menggunakan sebuah fungsi. Dari *state* yang di-*generate* di awal tadi, dihasilkan beberapa *state* generasi berikutnya melalui seleksi *parent*, *crossover*, dan mutasi.

Dalam tugas besar ini, kromosom direpresentasikan menggunakan State, dengan tiap kromosom berisi daftar kendaraan beserta atribut penempatannya (koordinat x dan y serta orientasi kendaraan horizontal atau vertikal). Satu kromosom merepresentasikan satu konfigurasi penempatan seluruh kendaraan di dalam kapal. Nilai kromosom dievaluasi menggunakan *fitness function*.

- **Fungsi getFitness**

```
   def getFitness(self):
        return self.getShippingFee() - 3 * self.getEtaPenalty()
```

Semakin tinggi nilai *fitness*, semakin baik konfigurasi penempatan yang dihasilkan. Algoritma berusaha memaksimalkan pendapatan pengiriman dengan mempertimbangkan urutan ETA.

Populasi awal dibentuk lewat runGeneticAlgorithm() menggunakan fungsi State.generateRandomState(). Jumlah individu ditentukan parameter populationSize. Setiap individu dibentuk lewat proses penempatan kendaraan secara acak. Urutan kendaraan yang akan ditempatkan diacak terlebih dahulu, kemudian setiap kendaraan dicoba untuk ditempatkan pada posisi dan orientasi yang dipilih secara acak.

Sebuah penempatan diterima jika tidak melanggar batas dimensi kapal, tidak bertabrakan dengan kendaraan lain, dan tidak menyebabkan total berat kendaraan yang ditempatkan melampaui kapasitas maksimum kapal. Apabila percobaan gagal, koordinat kendaraan dikosongkan dan proses dilanjutkan hingga batas percobaan tercapai.

Setelah populasi awal terbentuk, setiap individu dievaluasi menggunakan getFitness(). Kemudian individu yang memiliki nilai *fitness* tertinggi dipilih sebagai solusi terbaik awal. Program menyimpan nilai *fitness* tertinggi dan rata-rata populasi awal ke dalam variabel maxHistory dan meanHistory untuk memantau perkembangan algoritma.

- **Fungsi runGeneticAlgorithm**

```
   def runGeneticAlgorithm(self):
        startTime = time.perf_counter()
        population = [State.generateRandomState(self.ship,
self.vehicles) for _ in range(self.populationSize)]
        fitnesses = [s.getFitness() for s in population]
        best = population[fitnesses.index(max(fitnesses))]

        self.initialState = best.clone()
        self.maxHistory = [max(fitnesses)]
        self.meanHistory = [sum(fitnesses) / len(fitnesses)]
```

```
        for _ in range(self.generations):
            best = population[fitnesses.index(max(fitnesses))]
            nextPopulation = [best.clone()]
            while len(nextPopulation) < self.populationSize:
                x, y = self.select(population, fitnesses)
                child = self.recombine(x, y)
                nextPopulation.append(self.mutate(child))

            population = nextPopulation
            fitnesses = [s.getFitness() for s in population]
            self.maxHistory.append(max(fitnesses))
            self.meanHistory.append(sum(fitnesses) / len(fitnesses))

        self.iterations = self.generations
        self.duration = time.perf_counter() - startTime
        return population[fitnesses.index(max(fitnesses))]
```

Seleksi dilakukan lewat select() menggunakan fungsi random.choices() untuk memilih dua individu *random* sebagai *parent* untuk tahap *crossover*. Metode ini memberi peluang lebih besar bagi individu dengan nilai tinggi, tetapi tidak menutup kemungkinan bagi individu dengan nilai yang relatif rendah terpilih sebagai *parent*.

- **Fungsi select**

```
   def select(self, population, fitnesses):
        low = min(fitnesses)
        weights = [f - low + 1 for f in fitnesses]
        return random.choices(population, weights=weights, k=2)
```

*Crossover* dilakukan lewat recombine(), dimana sebuah titik potong dipilih secara *random* di antara indeks kendaraan. Kemudian, daftar kendaraan dari dua *parent*digabung. Bagian sebelum titik potong berasal dari *parent* pertama dan bagian setelah titik potong dari *parent* kedua. *Child* yang dihasilkan dibungkus sebagai State dan diproses lewat repair().

- **Fungsi recombine**

```
  def recombine(self, x, y):
        n = len(x.vehicles)
        point = random.randint(1, n-1)
        child = State(self.ship, x.vehicles[:point] +
y.vehicles[point:])
```

```
        return self.repair(child)
```

Mutasi dilakukan lewat mutate(), probabilitasnya ditentukan di parameter mutationRate (*default* 0.1). Jika mutasi terjadi, dipilih satu dari tiga macam operasi secara acak, swap, move, atau rotate.

- **Fungsi mutate**

```
  def mutate(self, state: State) -> State:
        if random.random() > self.mutationRate:
            return state

        n = len(state.vehicles)
        action = random.choice(["swap", "move", "rotate"])
        if action == "swap":
            a, b = random.sample(range(n), 2)
            state.swapVehiclePositions(a, b)
        else:
            v = random.randrange(n)
            if action == "rotate":
                state.rotateVehicle(v)
            else:
                w, l = state.vehicles[v].getCurrentDimensions()
                if w > self.ship.width or l > self.ship.length:
                    state.removeVehicle(v)
                else:
                    state.moveVehicle(v, random.randint(0,
self.ship.width - w), random.randint(0, self.ship.length - l))
        return self.repair(state)
```

Secara singkat, swap menukar koordinat dua kendaraan, move memindah kendaraan ke koordinat baru yang *random*, dan rotate mengubah orientasi kendaraan. Pada move, jika dimensi kendaraan melebihi kapal dalam orientasi sekarang maka kendaraan dikeluarkan. Jika tidak, akan coba dipindahkan ke posisi acak dalam batas dimensi kapal. Hasil mutasi kemudian akan diproses lewat repair().

- **Fungsi repair**

```
   def repair(self, state):
        positions = [(v.x, v.y) for v in state.vehicles]
        for v in state.vehicles:
            v.x = None
            v.y = None
        for v, (x, y) in zip(state.vehicles, positions):
```

```
            if x is not None:
                v.x = x
                v.y = y
            if not state.isValid():
                v.x = None
                v.y = None
        return state
```

Penanganan kromosom invalid adalah pekerjaan dari repair(). Hal ini diperlukan karena baik *crossover* maupun mutasi dapat menghasilkan konfigurasi yang tidak memenuhi batasan penempatan. Cara kerjanya adalah dengan menyimpan posisi awal seluruh kendaraan, mengosongkan koordinatnya, lalu memproses kendaraan satu persatu. Kendaraan yang punya koordinat sebelumnya akan dicoba untuk ditempatkan kembali di koordinat semula tersebut. Jika State menjadi tidak valid maka koordinatnya dikosongkan lagi. Pemeriksaan validitas utamanya mengecek total berat kendaraan tidak melebihi kapasitas maksimum kapal, seluruh kendaraan harus berada di dalam batas dimensi kapal, dan memastikan tidak ada kendaraan yang tumpang tindih.

Algoritma ini berhenti ketika telah menjalankan proses sebanyak jumlah generasi yang telah ditetapkan oleh parameter generations. Kemudian individu dengan nilai *fitness* tertinggi akan dikembalikan pada populasi terakhir. Pada setiap generasi, dilakukan pemantauan individu dengan nilai tertinggi sebagai berikut:

1. Memilih individu terbaik pada populasi saat ini
2. Menyalin individu terbaik ke populasi berikutnya (elitisme)
3. Menghasilkan individu baru melalui seleksi *parent*, *crossover*, dan mutasi hingga ukuran populasi terpenuhi
4. Mengevaluasi nilai *fitness* seluruh individu pada populasi baru.
5. Mencatat nilai *fitness* maksimum dan rata-rata populasi.

### 2.2.4 Visualisasi

Visualisasi hasil algoritma dibuat menggunakan HTML, CSS, JavaScript, dan library Chart.js.

Hasil yang ditampilkan terdiri dari:

1. Initial State
2. Final State
3. Plot nilai fitness terhadap iterasi atau generasi

Data hasil algoritma dikirim dari backend Flask dalam format JSON. Data tersebut berisi posisi kendaraan, nilai fitness, biaya pengiriman, penalti ETA, serta riwayat fitness.

- **Visualisasi Initial State dan Final State**

```
def buat_grid(state):
    grid = []
    for x in range(state.ship.width):
        grid.append([""] * state.ship.length)
    for kendaraan in state.vehicles:
        if kendaraan.x is None or kendaraan.y is None:
            continue
        lebar, panjang = kendaraan.getCurrentDimensions()
        for x in range(kendaraan.x, kendaraan.x + lebar):
            for y in range(kendaraan.y, kendaraan.y + panjang):
                grid[x][y] = kendaraan.id
    return grid
```

Hasil representasi grid dikirim sebagai initial_state_grid dan final_state_grid.

Di frontend, fungsi drawGrid() digunakan untuk menggambar grid secara dinamis menggunakan elemen HTML . Setiap sel kendaraan diberi warna berdasarkan class CSS masing-masing kendaraan.

```
function drawGrid(id, data) {
    var container = document.getElementById(id);
    if (!container) return;
    container.innerHTML = "";
    var grid = document.createElement("div");
    grid.className = "ship-grid";
    for (var i = 0; i < data.length; i++) {
        for (var j = 0; j < data[i].length; j++) {
            var cell = document.createElement("div");
            var value = data[i][j];
            cell.className = value
                ? "grid-cell filled"
                : "grid-cell empty-cell";
            cell.innerText = value || "";
            grid.appendChild(cell);
        }
    }
    container.appendChild(grid);
}
```

Fungsi ini dipanggil untuk menampilkan kondisi awal dan kondisi akhir:

```
drawGrid("initial-state-container", result.initial_state_grid);
drawGrid("final-state-container", result.final_state_grid);
```

- **Visualisasi Plot**

Plot dibuat menggunakan library Chart.js. Library ini digunakan untuk menampilkan grafik berbentuk garis pada elemen HTML <canvas>. Fungsi chart() pada JavaScript membuat grafik berdasarkan data riwayat fitness yang dikirim dari backend.

```
function chart(id, datasets) {
    var canvas = document.getElementById(id);
    if (!canvas) return;

    charts[id] = new Chart(canvas, {
        type: "line",
        data: {
            labels: labels,
            datasets: datasets
        },
        options: {
            responsive: true,
            maintainAspectRatio: false
        }
    });
}
```

Untuk Hill Climbing dan Simulated Annealing, grafik menampilkan nilai objective function terhadap iterasi:

```
chart("fitness-chart", [
    {
        label: "Objective Function",
        data: result.fitness_history,
        borderColor: "#176b87",
        fill: false
    }
]);
```

Untuk Genetic Algorithm, grafik menampilkan fitness maksimum dan fitness rata-rata terhadap generasi:

```
chart("ga-chart", [
    {
        label: "Maksimum",
        data: extra.max_history,
        borderColor: "#176b87",
        fill: false
    },
    {
        label: "Rata-rata",
        data: extra.mean_history,
        borderColor: "#f28482",
        fill: false
    }
]);
```

## 2.3 Hasil dan Analisis Eksperimen

### 2.3.1 Setup Eksperimen

Eksperimen dilakukan untuk mengevaluasi performa algoritma pencarian lokal dalam mengoptimalkan penataan kendaraan pada geladak kapal. Seluruh varian algoritma diuji menggunakan dataset dan *environment* yang seragam untuk menjaga transparansi dan keadilan hasil pengujian.

*Instance* Masalah:

- Ukuran Geladak Kapal: Grid berukuran 7x7
- Kapasitas Maksimum Kapal: 15
- Kendaraan: 7 kendaraan dengan spesifikasi;

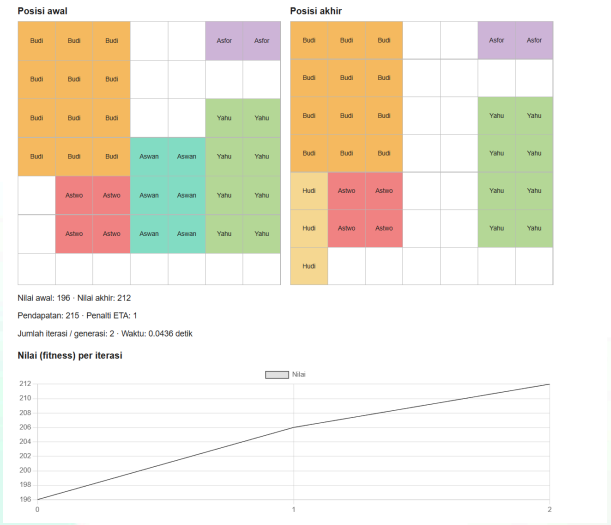
| ID Kendaraan | Dimensi | Shipping Fee | Berat | ETA |
|---|---|---|---|---|
| **Budi** | 3 × 4 | 60 | 5 | 3 |
| **Aswan** | 2 × 3 | 40 | 3 | 1 |
| **Astwo** | 2 × 2 | 25 | 2 | 5 |
| **Astri** | 4 × 4 | 90 | 8 | 2 |

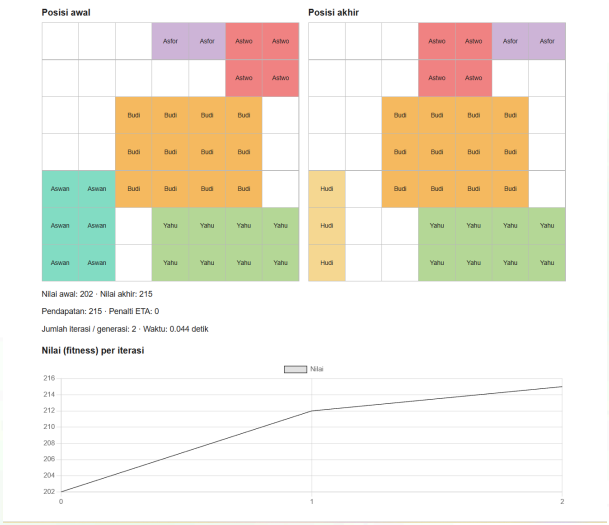
| | | | | |
|---|---|---|---|---|
| **Asfor** | 1 × 2 | 10 | 1 | 7 |
| **Yahu** | 4 × 2 | 70 | 4 | 4 |
| **Hudi** | 1 × 3 | 50 | 3 | 2 |

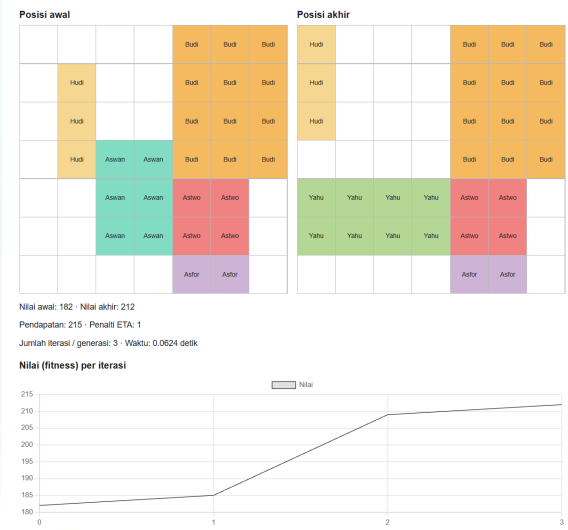
### 2.3.2 Hill-Climbing: Steepest Ascent

Eksperimen Steepest Ascent dijalankan dengan mengevaluasi seluruh neighbors yang valid pada setiap iterasi dan selalu memilih neighbor dengan kenaikan fitness score paling curam/tertinggi. Algoritma langsung berhenti ketika tidak ditemukan neighbor yang memiliki skor lebih tinggi dari state saat ini (*local maximum*).

**Hasil Eksperimen:**






Tabel Rangkuman Hasil Steepest Ascent Hill-Climbing:

| Run | Nilai Awal | Nilai Akhir | Iterasi | Durasi |
|---|---|---|---|---|
| 1 | 196 | 212 | 2 | 0.0436 |
| 2 | 202 | 215 | 2 | 0.044 |
| 3 | 182 | 212 | 3 | 0.0624 |

Ketiga run berhenti dengan cepat, hanya dalam 2-3 iterasi. Pada setiap iterasi, Steepest Ascent akan memilih neighbor terbaik dari seluruh neighbor yang valid, sehingga nilainya naik tajam di awal (+16, +13, dan +30) lalu cepat mencapai local maximum. Nilai akhirnya cukup konsisten, yaitu 212, 215, dan 212 (rata-rata 213), walaupun nilai awalnya berbeda cukup jauh (182 sampai 202). Run 2 mencapai 215, lebih tinggi dari 212 pada run lainnya, 215 juga bukan global maximum. Steepest Ascent tidak bisa keluar dari local maximum (tanpa *sideways move* maupun *restart*), jadi hasil akhirnya sangat bergantung pada state awal acak.

Durasi pencarian sangat singkat, sekitar 0.04~0.06 detik per run. Durasinya terlihat sedikit naik semakin menambah iterasi (run 3 dengan 3 iterasi paling lama), karena setiap iterasi harus menyusun dan mengevaluasi seluruh *neighbor valid*, dan satu pemindaian tambahan dibutuhkan di akhir untuk memastikan tidak ada neighbor yang lebih baik.

### 2.3.3 Simulated Annealing

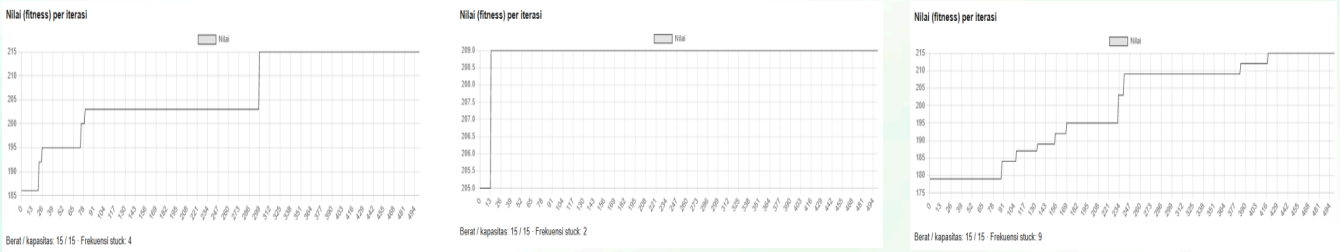
Eksperimen Simulated Annealing dijalankan dengan memilih satu neighbor acak yang valid pada setiap iterasi dan mengevaluasi perubahan nilai fitness (Delta E). Jika neighbor memiliki skor lebih tinggi, langkah langsung diterima, sedangkan jika skornya lebih rendah, langkah tetap dapat diterima berdasarkan probabilitas kriteria Metropolis (e^Delta E / T) dengan batas penerimaan di angka 0.5. Proses pencarian dilakukan sebanyak 500 iterasi dengan penurunan suhu secara bertahap. Algoritma mencatat frekuensi stuck ketika menemukan kondisi di mana nilai skor tidak mengalami peningkatan (stuck) berturut-turut lebih dari 10 kali.

**Hasil Eksperimen:**

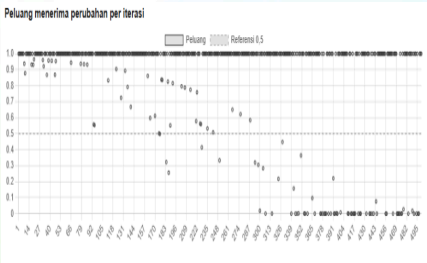
1. Hasil awal dan akhir grid

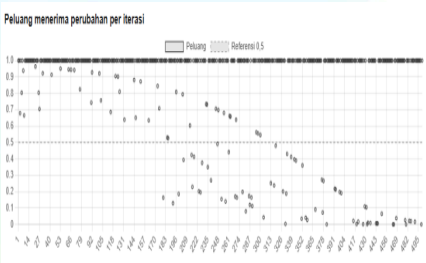


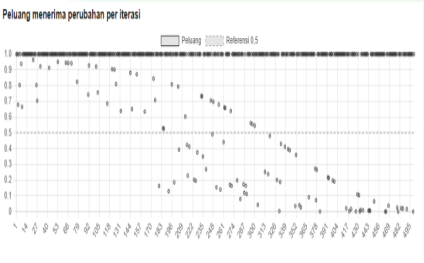

2. Nilai fitness per iterasi




3. Peluang menerima perubahan per iterasi








Tabel Rangkuman Hasil Simulated Annnealing:

| Run | Nilai Awal | Nilai Akhir | Frekuensi Stuck | Durasi |
|---|---|---|---|---|
| 1 | 186 | 215 | 4 | 0.115 |
| 2 | 205 | 209 | 2 | 0.1036 |
| 3 | 179 | 215 | 9 | 0.099 |

Nilai akhir yang diperoleh adalah 215 pada Run 1, 209 pada Run 2, dan 215 pada Run 3, dengan rata-rata sebesar 213 dari rentang nilai awal 179 hingga 205. Pencapaian nilai akhir 215 pada Run 1 dan Run 3 menunjukkan bahwa penerimaan perpindahan ke state dengan skor lebih rendah di awal iterasi berhasil mencegah algoritma berhenti prematur pada posisi lokal. Sebaliknya, Run 2 dengan nilai awal tertinggi (205) justru berakhir di angka 209, yang mengindikasikan bahwa state awal yang lebih tinggi tidak menjamin pencapaian solusi akhir yang lebih optimal.

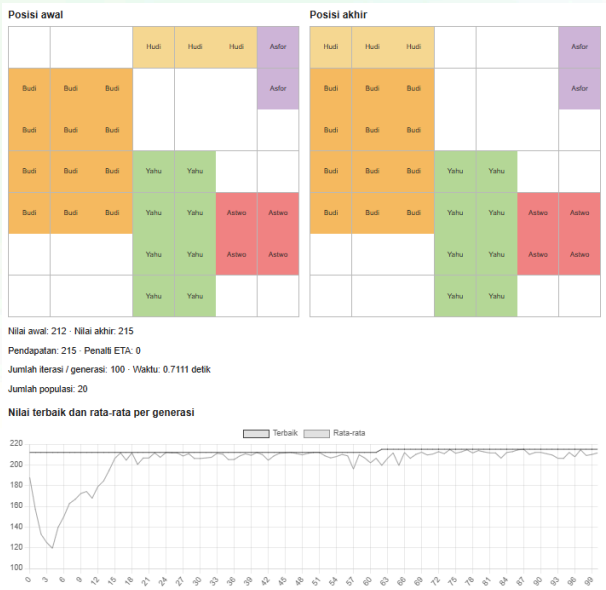
Frekuensi stuck mencatatkan angka 4 kali pada Run 1, 2 kali pada Run 2, dan 9 kali pada Run 3. Jumlah stuck terbanyak pada Run 3 berkaitan dengan nilai awal terendah (179), yang menyebabkan algoritma melakukan lebih banyak penerimaan acak di awal sebelum akhirnya menemukan jalur peningkatan menuju nilai akhir 215. Durasi eksekusi pencarian mencatatkan waktu 0.115 detik pada Run 1, 0.1036 detik pada Run 2, dan 0.099 detik pada Run 3.

### 2.3.4 Genetic Algorithm

Eksperimen Genetic Algorithm dijalankan dengan membagi pengujian menjadi dua skenario. Yang pertama adalah jumlah generasi tetap (100) dengan ukuran populasi berbeda-beda (20, 50, 100), sementara yang kedua dengan ukuran populasi tetap (100) dan jumlah generasi berbeda-beda (10, 30, 80). Angka-angka ini sesuai dengan variabel yang telah ditetapkan dalam GA_CONFIGS pada ga.py. Setiap

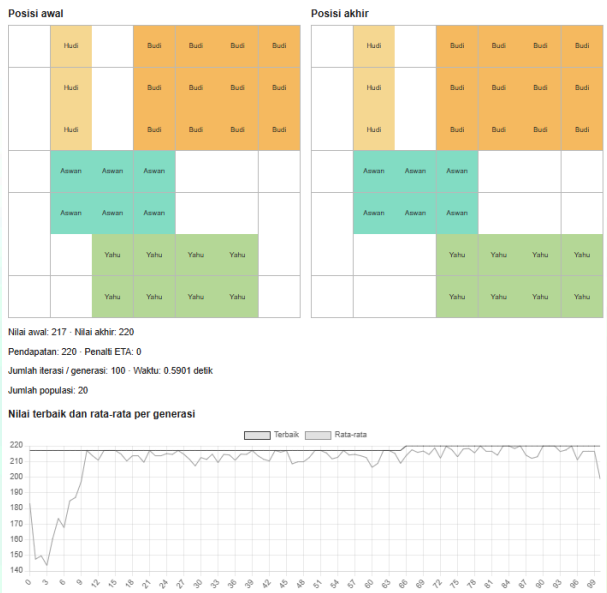
konfigurasi dijalankan sebanyak tiga kali untuk mengamati variasi hasil akibat proses acak dalam inisialisasi populasi, seleksi *parent*, *crossover*, dan mutasi.

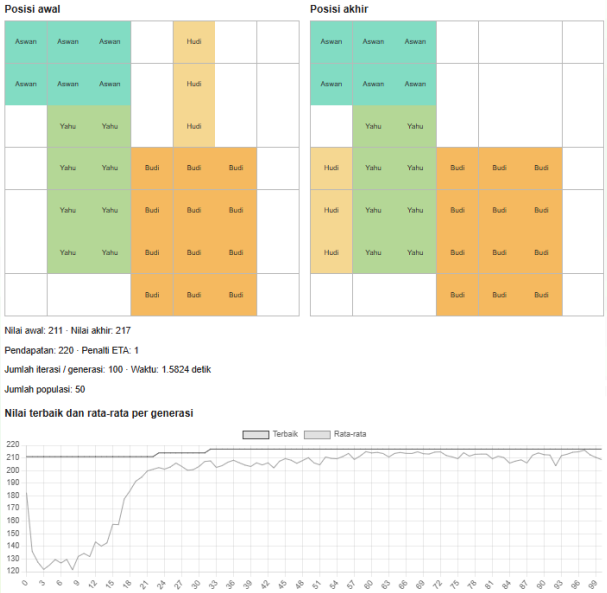
**a. Kontrol populasi, variasi iterasi (generasi)** (3 variasi x 3 run)



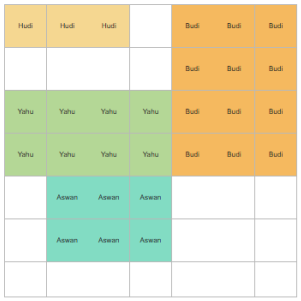




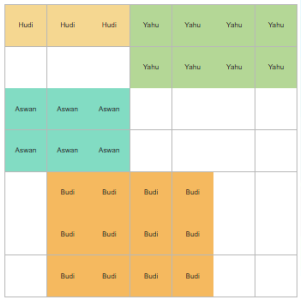


**Posisi awal**




**Posisi akhir**




Nilai awal: 211 · Nilai akhir: 220

Pendapatan: 220 · Penalti ETA: 0

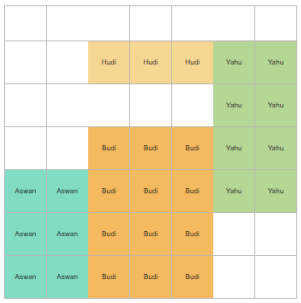
Jumlah iterasi / generasi: 100 · Waktu: 1.4502 detik

Jumlah populasi: 50

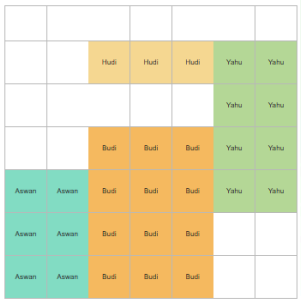
**Nilai terbaik dan rata-rata per generasi**




**Posisi awal**




**Posisi akhir**



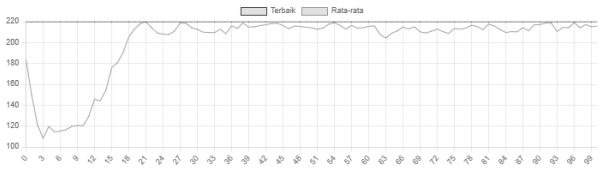

Nilai awal: 220 · Nilai akhir: 220

Pendapatan: 220 · Penalti ETA: 0

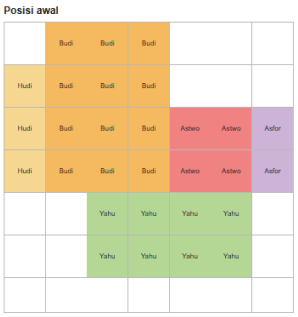
Jumlah iterasi / generasi: 100 · Waktu: 1.4913 detik

Jumlah populasi: 50

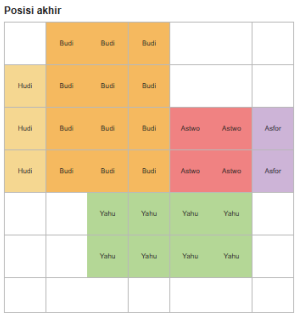
**Nilai terbaik dan rata-rata per generasi**




**Posisi awal**




**Posisi akhir**



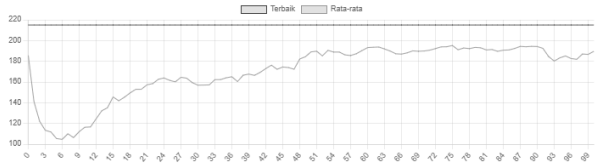

Nilai awal: 215 · Nilai akhir: 215

Pendapatan: 215 · Penalti ETA: 0

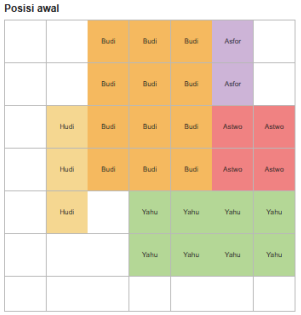
Jumlah iterasi / generasi: 100 · Waktu: 3.0481 detik

Jumlah populasi: 100

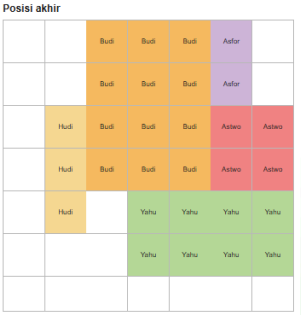
**Nilai terbaik dan rata-rata per generasi**




**Posisi awal**




**Posisi akhir**



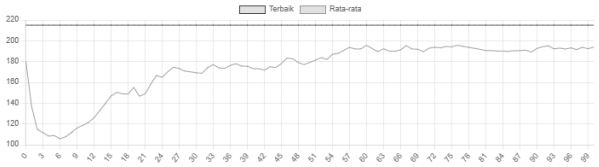

Nilai awal: 215 · Nilai akhir: 215

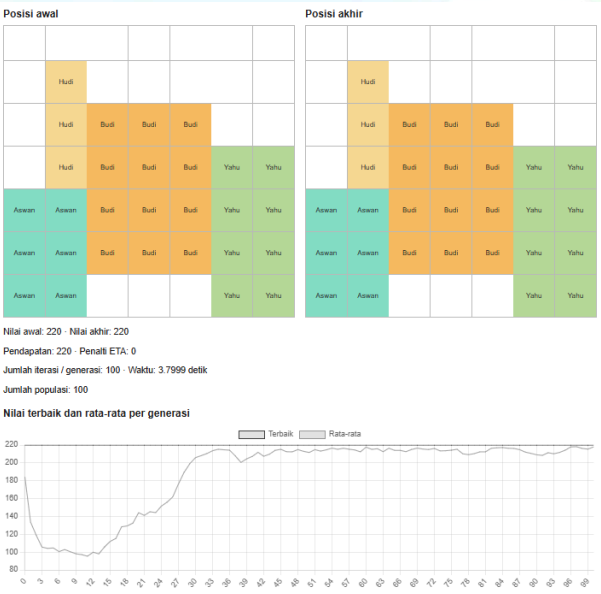
Pendapatan: 215 · Penalti ETA: 0

Jumlah iterasi / generasi: 100 · Waktu: 3.6 detik

Jumlah populasi: 100

**Nilai terbaik dan rata-rata per generasi**

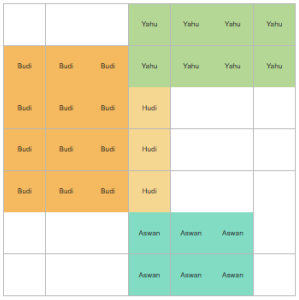




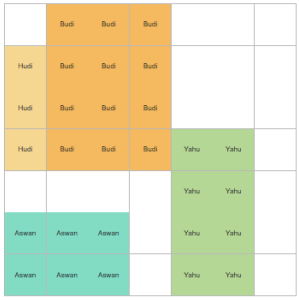

| Populasi | Iterasi | Run | Nilai Akhir | Durasi |
|---|---|---|---|---|
| 20 | 100 | 1 | 215 | 0.7111 |
| | | 2 | 212 | 0.6186 |
| | | 3 | 220 | 0.5901 |
| 50 | | 1 | 217 | 1.5824 |
| | | 2 | 220 | 1.4502 |
| | | 3 | 220 | 1.4913 |
| 100 | | 1 | 215 | 3.0481 |
| | | 2 | 215 | 3.6 |
| | | 3 | 220 | 3.7999 |

**b. Kontrol iterasi, variasi populasi** (3 variasi x 3 run)

Posisi awal




Posisi akhir



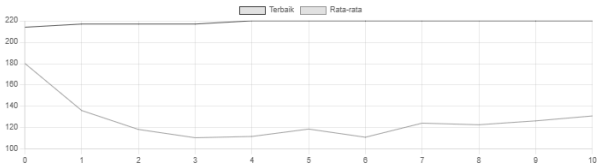

Nilai awal: 214 · Nilai akhir: 220

Pendapatan: 220 · Penalti ETA: 0

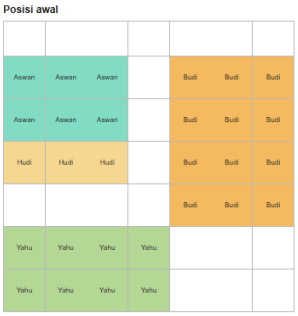
Jumlah iterasi / generasi: 10 · Waktu: 0.8403 detik

Jumlah populasi: 100

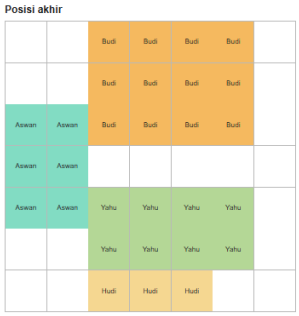
**Nilai terbaik dan rata-rata per generasi**




Posisi awal




Posisi akhir



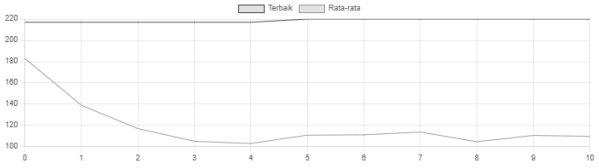

Nilai awal: 217 · Nilai akhir: 220

Pendapatan: 220 · Penalti ETA: 0

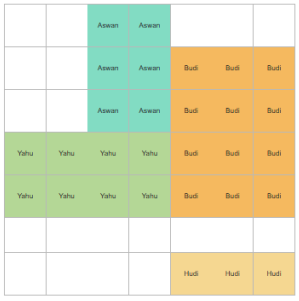
Jumlah iterasi / generasi: 10 · Waktu: 0.594 detik

Jumlah populasi: 100

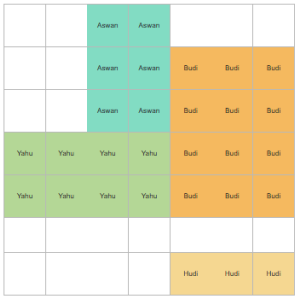
**Nilai terbaik dan rata-rata per generasi**




Posisi awal




Posisi akhir



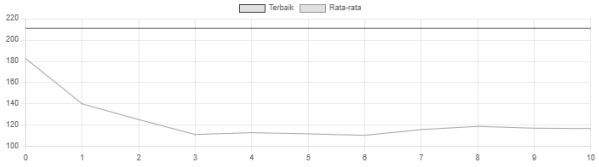

Nilai awal: 211 · Nilai akhir: 211

Pendapatan: 220 · Penalti ETA: 3

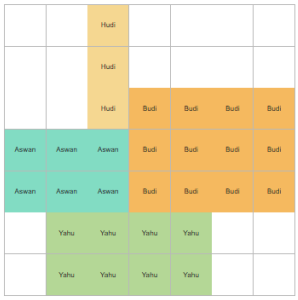
Jumlah iterasi / generasi: 10 · Waktu: 0.7814 detik

Jumlah populasi: 100

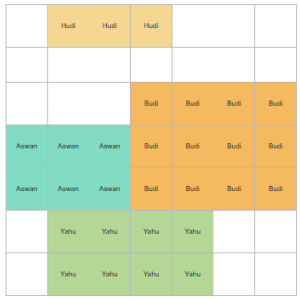
**Nilai terbaik dan rata-rata per generasi**




Posisi awal




Posisi akhir



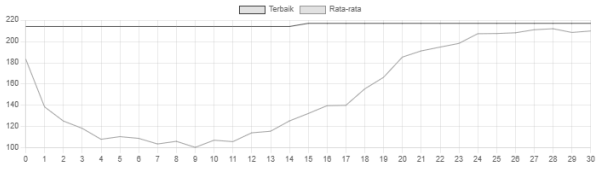

Nilai awal: 214 · Nilai akhir: 217

Pendapatan: 220 · Penalti ETA: 1

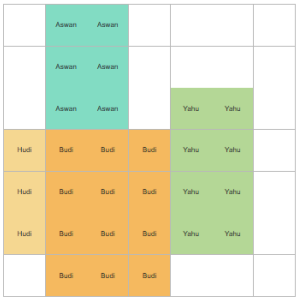
Jumlah iterasi / generasi: 30 · Waktu: 0.8122 detik

Jumlah populasi: 100

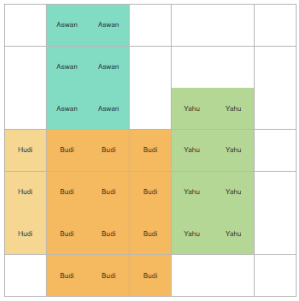
**Nilai terbaik dan rata-rata per generasi**



Posisi awal




Posisi akhir



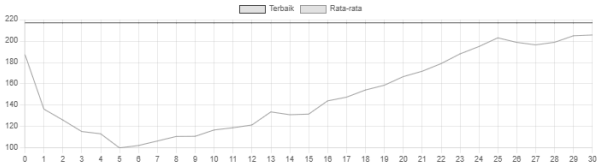

Nilai awal: 217 · Nilai akhir: 217

Pendapatan: 220 · Penalti ETA: 1

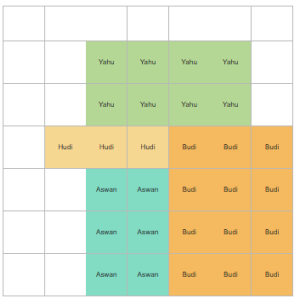
Jumlah iterasi / generasi: 30 · Waktu: 1.1113 detik

Jumlah populasi: 100

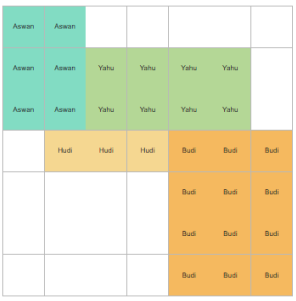
Nilai terbaik dan rata-rata per generasi




Posisi awal




Posisi akhir



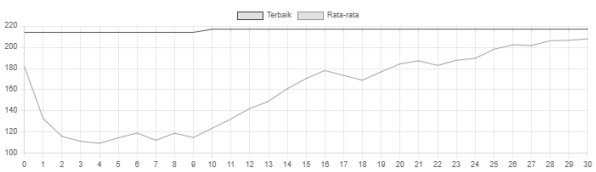

Nilai awal: 214 · Nilai akhir: 217

Pendapatan: 220 · Penalti ETA: 1

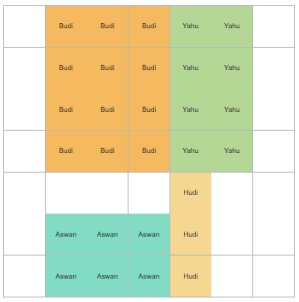
Jumlah iterasi / generasi: 30 · Waktu: 0.8609 detik

Jumlah populasi: 100

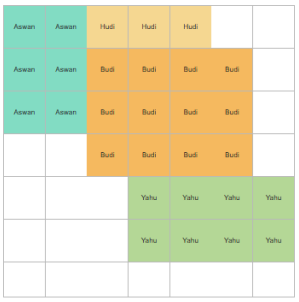
Nilai terbaik dan rata-rata per generasi




Posisi awal




Posisi akhir



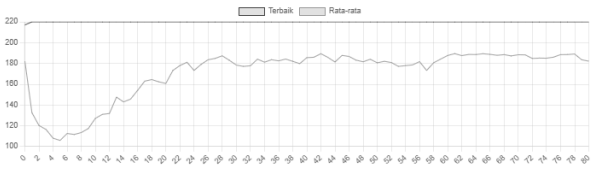

Nilai awal: 217 · Nilai akhir: 220

Pendapatan: 220 · Penalti ETA: 0

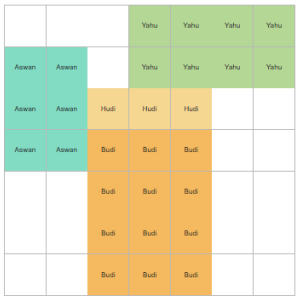
Jumlah iterasi / generasi: 80 · Waktu: 2.8688 detik

Jumlah populasi: 100

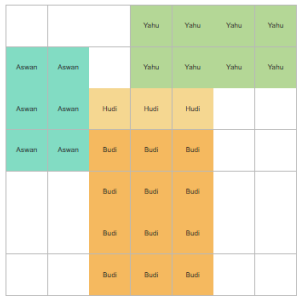
Nilai terbaik dan rata-rata per generasi




Posisi awal




Posisi akhir



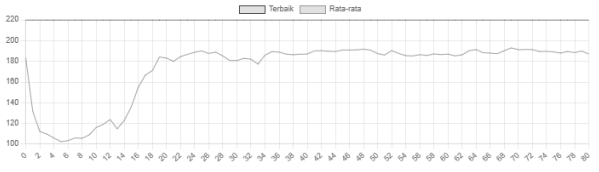

Nilai awal: 220 · Nilai akhir: 220

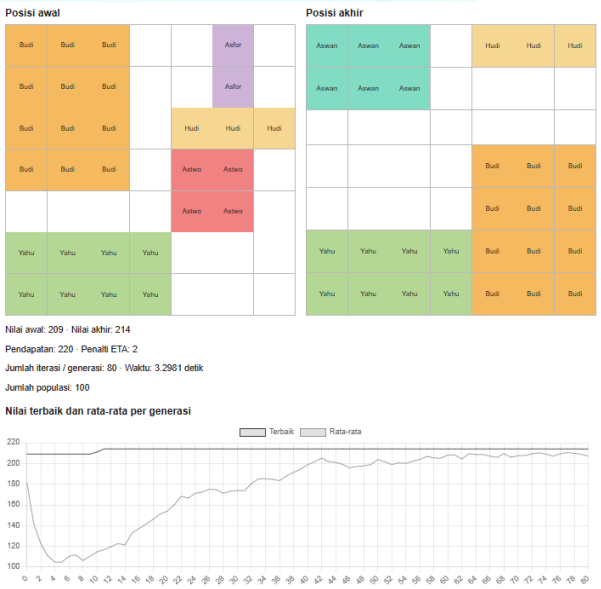
Pendapatan: 220 · Penalti ETA: 0

Jumlah iterasi / generasi: 80 · Waktu: 3.1154 detik

Jumlah populasi: 100

Nilai terbaik dan rata-rata per generasi






| Populasi | Iterasi | Run | Nilai Akhir | Durasi |
|---|---|---|---|---|
| 100 | 10 | 1 | 220 | 0.8403 |
| | | 2 | 220 | 0.594 |
| | | 3 | 211 | 0.7814 |
| | 30 | 1 | 217 | 0.8122 |
| | | 2 | 217 | 1.1113 |
| | | 3 | 217 | 0.8609 |
| | 80 | 1 | 220 | 2.8688 |
| | | 2 | 220 | 3.1154 |
| | | 3 | 214 | 3.2981 |

Setiap konfigurasi dijalankan tiga kali, dengan rincian nilai akhir dan durasi dalam detik tertera pada tabel di atas. Nilai *fitness* diperoleh dari *shipping fee* dikurangi 3 x penalti ETA, untuk penalti ETA sendiri didapat dari jumlah pasangan kendaraan yang diangkut ke kapal dengan urutan ETA terbalik terhadap posisinya pada sumbu y. Pada visualisasi, sumbu horizontal menyatakan koordinat y dan sumbu vertikal menyatakan koordinat x. Penalti ETA 0 tercapai jika kendaraan dengan ETA lebih kecil berada di sisi kiri (y lebih kecil) dibanding kendaraan ber-ETA lebih besar. *State* awal yang

ditampilkan merupakan individu dengan nilai tertinggi pada populasi awal, sementara *state* akhir merupakan individu dengan nilai tertinggi pada populasi generasi terakhir. Individu terbaik tiap generasi akan selalu dibawa ke generasi berikutnya, atau disebut juga elitisme.

Nilai *global optimum* di sini ialah *shipping fee*=220, dengan kombinasi pengangkutan kendaraan dengan *shipping fee* tertinggi dan berat total tidak melebihi kapasitas kapal ialah {Aswan, Budi, Hudi, Yahu} yang memiliki *fee* 220 dan total berat 15. Dengan *genetic algorithm*, *global optimum* tercapai dalam 8 dari 18 eksperimen yang dijalankan. Rata-rata nilai akhir seluruh percobaan adalah 217.2 dengan nilai terendah 211. Beberapa penyebab mengenai *shipping fee* yang berakhir di bawah *global optimum* adalah penalti ETA dan terjebak di *local optimum* batas kapasitas berat. Untuk penalti ETA, ada beberapa pasang kendaraan yang urutannya terbalik terhadap posisi y. Sementara untuk percobaan yang *stuck* di *local optimum*, ada beberapa kombinasi yang total beratnya sudah mencapai 15, sehingga tidak valid untuk menukar kendaraan yang sudah diangkut dengan kendaraan di luar kapal yang mengakibatkan total berat melebihi kapasitas kapal. Karena algoritma ini cenderung mempertahankan individu dengan nilai yang tinggi, alur melalui nilai yang lebih rendah jarang dilalui. Untuk kendaraan dengan ID Astri tidak terangkut dalam percobaan karena kombinasi terbaik yang mengangkutnya bernilai 210 saja, dengan kombinasi {Astri, Hudi, Yahu}.

Eksperimen dengan jumlah iterasi atau generasi tetap (100) dengan variasi pada ukuran populasi memiliki rata-rata nilai akhir 215.7, 219, dan 216.7 untuk ukuran 20, 50, dan 100 secara berurutan. Di sini, dapat dilihat bahwa populasi 50 memberikan nilai terbaik dibandingkan kedua ukuran populasi lainnya, dengan populasi 100 tidak jauh berbeda dengan populasi 20. Populasi yang besar memberikan ruang untuk perbaikan lebih besar dibanding populasi berukuran kecil. Populasi awal yang besar memberikan kesempatan untuk beberapa *state* yang sedari awal memiliki nilai yang tinggi untuk muncul dan terpilih ke generasi berikutnya, sehingga perbaikan dapat dilakukan lebih cepat. Perubahan durasi juga tampak naik sebanding dengan kenaikan ukuran populasi.

Eksperimen dengan jumlah populasi tetap (100) dengan variasi iterasi sebanyak 10, 30, dan 80 memiliki nilai akhir yang rata-ratanya berkisar antara 217 hingga 218. Tidak menunjukkan tren yang monoton dan belum cukup untuk menyimpulkan bahwa jumlah iterasi dapat mempengaruhi hasil secara signifikan dengan hanya tiga kali percobaan per konfigurasi. Pengaruh yang jelas terletak pada durasi. Semakin banyak jumlah iterasi, semakin lama durasinya. Durasi per kombinasi relatif stabil untuk semua percobaan, dengan kenaikan ukuran populasi dan/atau banyak iterasi menjadikan durasi lebih lama.

Pada algoritma ini, kurva umumnya mengalami penurunan tajam di awal akibat *crossover* dua *parent* secara acak. Baru proses *repair* kemudian menyaring hasil yang dianggap melanggar aturan dan seleksi

kemudian menaikkan rata-rata. Keterbatasan dari percobaan dengan algoritma ini ialah percobaan yang hanya dijalankan tiga kali per konfigurasi menjadikan hasil percobaan kurang cukup untuk menarik kesimpulan yang tegas.

### 2.3.5 Analisis Perbandingan Antar Algoritma

**a. Seberapa dekat tiap algoritma ke global optimum dan mengapa.**

Pada instance ini global optimum dapat ditentukan, yaitu 220. **Nilai 220 dicapai oleh kombinasi Budi, Aswan, Yahu, dan Hudi (total berat 15, total fee 220) dengan penalti ETA 0, sehingga global optimum instance ini adalah 220.**

Hill-Climbing mencapai rata-rata 213 atau sekitar 96,8% dari optimum, dengan nilai terbaik 215 dan tidak ada run yang mencapai 220. Hal ini terjadi karena algoritma hanya menerima tetangga yang lebih baik sehingga berhenti di local maximum (212 atau 215) dalam 2-3 iterasi, dan hasil akhirnya sangat bergantung pada state awal acak.

Simulated Annealing juga rata-rata 213 (96,8%), dengan dua run mencapai 215 dan satu run hanya 209. Penerimaan langkah buruk di awal iterasi membantu keluar dari local optimum, tetapi dengan satu tetangga acak per iterasi dan jumlah iterasi yang terbatas, ruang pencarian belum tereksplorasi cukup untuk mencapai 220.

Genetic Algorithm adalah satu-satunya algoritma yang mencapai optimum, dengan rata-rata 217,22 (98,7%) dan 8 dari 18 run mencapai 220. Keunggulan ini disebabkan oleh populasi yang mengeksplorasi banyak solusi secara paralel, crossover yang menggabungkan potongan penempatan baik dari dua parent, serta elitisme yang menjamin nilai terbaik tidak pernah turun.

**b. Perbandingan hasil antar algoritma.**

Hill-Climbing dan Simulated Annealing sama-sama berakhir di rata-rata 213, tetapi dengan karakter berbeda. Hill-Climbing stabil (212, 215, 212) namun tidak mampu keluar dari local maximum, sedangkan Simulated Annealing lebih berpeluang melewatinya, terbukti dari dua run yang mencapai 215, tetapi hasilnya lebih bervariasi (209 sampai 215). Genetic Algorithm unggul baik pada rata-rata maupun nilai terbaik. Konfigurasi terbaiknya, yaitu populasi 50 dengan 100 generasi, menghasilkan rata-rata 219 dengan 2 dari 3 run mencapai 220.

**c. Perbandingan durasi pencarian.**

Hill-Climbing paling cepat dengan rata-rata sekitar 0,050 detik per run karena hanya membutuhkan 2-3 iterasi. Simulated Annealing membutuhkan rata-rata sekitar 0,106 detik, kira-kira dua kali lebih lama, karena menjalankan ratusan iterasi meskipun setiap iterasinya cepat. Genetic Algorithm paling lambat: rata-rata sekitar 0,640 detik pada populasi 20, 1,508 detik pada populasi 50, dan 3,483 detik pada populasi 100 (dengan 100 generasi). Hal ini terjadi karena setiap generasi mengevaluasi seluruh populasi, termasuk perhitungan penalti ETA berpasangan, serta menjalankan `repair()`. Pada konfigurasi terbesar, GA sekitar 70 kali lebih lambat daripada Hill-Climbing, meskipun secara absolut selisihnya hanya beberapa detik untuk 7 kendaraan.

Secara keseluruhan, Hill-Climbing paling cocok bila kecepatan menjadi prioritas, Simulated Annealing menawarkan keseimbangan antara kecepatan dan kemampuan menghindari local optimum, dan Genetic Algorithm memberikan kualitas solusi terbaik dengan biaya waktu komputasi tertinggi.

# BAB 3
# Kesimpulan dan Saran

## 3.1 Kesimpulan

1. Telah dirancang sebuah fungsi objektif yang dapat menilai kualitas suatu solusi penempatan kendaraan di dek kapal secara kuantitatif.
2. Telah diimplementasikan 3 algoritma *local search* untuk menyelesaikan masalah optimasi penempatan kendaraan di dek kapal.
3. Telah dibandingkan performa setiap algoritma yang digunakan jika dibandingkan dengan algoritma lain yang digunakan.

## 3.2 Saran

1. Bereksperimen dengan variabel temperatur, threshold penerimaan, dan penurunan suhu pada algoritma Simulated Annealing.
2. Menggunakan varian lain untuk Hill-Climbing

# BAB 4
# Pembagian Tugas

| Nama | NIM | Kontribusi |
|---|---|---|
| Amanda Trishya Aulia | 18224020 | - Mengerjakan Implementasi Algoritma Genetic Algorithm<br>- Mengerjakan Laporan |
| Mineva Azzahra | 18224029 | - Mengerjakan Implementasi Algoritma Hill-Climbing<br>- Mengerjakan Laporan |
| Adriel Nathanael Simatupang | 18224109 | - Mengerjakan Implementasi Algoritma Simulated Annealing<br>- Mengerjakan Laporan |

# Referensi


- Python Software Foundation, “Python 3 Documentation.” [Online]. Available: https://docs.python.org/3/
- Wikipedia, “Genetic algorithm.” [Online]. Available: https://en.wikipedia.org/wiki/Genetic_algorithm
- Slide kuliah IF3070, Beyond Classical Search

- Sumber internet di source code:
  https://github.com/aimacode/aima-python/blob/master/search.py
  https://github.com/hellerbarde/BinPack/blob/master/binpack/item.py

# Lampiran

- Link repository GitHub IF3070_G41_papoi , release v1.0
- Form Penggunaan GenAI

**FORMAT DEKLARASI PENGGUNAAN AI**
**SURAT PERNYATAAN PENGGUNAAN AI**

Dengan ini, saya/kami*:

| | | |
|---|---|---|
| Nama/NIM | : | Amanda T.A./18224020, Mineva A./18224029, Adriel N. S./18224109 |
| Fakultas/Sekolah | : | STEI-K |
| Kode Mata Kuliah | : | IF3070 |
| Nama Mata Kuliah | : | Dasar Intelegensi Artifisial |
| Judul Tugas/Proposal | : | Penempatan Kendaraan pada Geladak Kapal dengan Algoritma Local Search |

Menyatakan bahwa saya/kami* menggunakan/tidak menggunakan* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

| Lingkup Pekerjaan | Digunakan? | Tingkat Penggunaan |
|---|---|---|
| **Pemeriksaan Ejaan:** menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya. | Ya/Tidak* | - |
| **Pembuatan Teks:** menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya. | Ya/Tidak* | - |
| **Bantuan Diskusi dalam Penyusunan Konten:** menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya. | Ya/Tidak* | - |
| **Pembuatan Gambar, Video, dan/atau Grafik:** menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya. | Ya/Tidak* | - |
| **Lainnya, Jelaskan:** | | |

Saya/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 22 Oktober 2026



Mineva Azzahra
18224029